

Nexus_Framework_Deconstructs_Aging_Research.project_companion_note

March 27, 2026

1 Nexus Framework Deconstructs Aging Research — Companion Notebook

This notebook operationalizes the aging paper through the project's shared operator floor:

- identity $\rightarrow$ gap $\rightarrow$ closure
- value channel + shape channel
- folding as the same executable grammar across biology, protein structure, and hashing
- localized reset, protocol mismatch, and stability limits around $H = \pi/9$

The notebook is organized as a paper-facing appendix rather than a general project dump.

1.1 Imports and core constants

The notebook uses only offline, local computation.
No network access is required.

```
[1]: import math
import hashlib
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from scipy.stats import pearsonr
from mpl_toolkits.mplot3d import Axes3D # noqa: F401

H = np.pi / 9
M32 = 0xFFFFFFFF

np.set_printoptions(suppress=True, precision=4)
pd.set_option("display.max_columns", 20)
pd.set_option("display.width", 140)
```

1.2 1. Operator floor: kernel triad, closure, and the biological carry channel

The project documents treat the same event as three simultaneous reads:

- **path / curvature**
- **residue / carry / shape**
- **projection / observable**

For binary words, the irreducible triad is

$$[x + y = (x \oplus y) + 2(x \& y)]$$

The XOR term is the fast visible path.

The shifted AND term is the slow historical residue.

The sum is the observed child.

```
[2]: def kernel_triad_word(x: int, y: int):
    xor_term = (x ^ y) & M32
    and2_term = ((x & y) << 1) & M32
    summed = (x + y) & M32
    return xor_term, and2_term, summed

def popcount32(x: int) -> int:
    return int(x & M32).bit_count()

def norm32(x: int) -> float:
    return popcount32(x) / 32.0

demo_pairs = [
    ("baseline", 0x6A09E667, 0x243F6A88),
    ("shape-biased", 0x6A09E667, 0x9E3779B9),
    ("damage-event", 0xA5A5A5A5, 0x0F0F33CC),
    ("anti-state", 0x89ABCDEF, 0x76543210),
]

rows = []
for label, x, y in demo_pairs:
    xor_term, and2_term, summed = kernel_triad_word(x, y)
    rows.append({
        "label": label,
        "x": f"0x{x:08x}",
        "y": f"0x{y:08x}",
        "xor": f"0x{xor_term:08x}",
        "and2": f"0x{and2_term:08x}",
        "sum": f"0x{summed:08x}",
        "xor_density": norm32(xor_term),
        "and2_density": norm32(and2_term),
        "sum_density": norm32(summed),
    })
```

```

triad_df = pd.DataFrame(rows)
triad_df

```

```

[2]:      label      x      y      xor      and2      sum
xor_density and2_density sum_density
0      baseline 0x6a09e667 0x243f6a88 0x4e368cef 0x4012c400 0x8e4950ef
0.5625      0.18750      0.50000
1  shape-biased 0x6a09e667 0x9e3779b9 0xf43e9fde 0x1402c042 0x08416020
0.6875      0.21875      0.18750
2  damage-event 0xa5a5a5a5 0x0f0f33cc 0xaaaa9669 0x0a0a4308 0xb4b4d971
0.5000      0.25000      0.53125
3   anti-state 0x89abcdef 0x76543210 0xffffffff 0x00000000 0xffffffff
1.0000      0.00000      1.00000

```

1.3 2. Aging as accumulated carry history

To make the paper’s claim executable, we treat cellular history as repeated local collisions:

- incoming event
- current state
- visible update
- carry exhaust

The visible channel is the current phenotype-like state.

The accumulated carry channel is the surrogate epigenomic history.

```

[3]: def schedule_events(n_steps: int, lesion_start=70, lesion_end=120, seed=7):
    rng = np.random.default_rng(seed)
    events = []
    for t in range(n_steps):
        base = rng.integers(0, 2**32, dtype=np.uint32).item()
        if lesion_start <= t < lesion_end:
            base ^= 0x00FF00FF
        if t % 19 == 0:
            base ^= 0x0F0F0F0F
        events.append(int(base))
    return events

def simulate_carry_history(n_steps=180, reset_step=None, reset_gain=H,
    anti_mode="carry_only"):
    state = 0x6A09E667
    carry_trace = 0
    visible_density = []
    carry_density = []
    clock = []
    states = []

    events = schedule_events(n_steps)

```

```

for t, ev in enumerate(events):
    xor_term, and2_term, summed = kernel_triad_word(state, ev)
    carry_trace ^= and2_term
    state = summed

    if reset_step is not None and t == reset_step:
        if anti_mode == "carry_only":
            carry_trace = int(((1.0 - reset_gain) * carry_trace)) & M32
        elif anti_mode == "state_and_carry":
            carry_trace = int(((1.0 - reset_gain) * carry_trace)) & M32
            state = int(((1.0 - reset_gain) * state)) & M32
        elif anti_mode == "hard_flip":
            carry_trace = 0
            state = 0

    visible_density.append(norm32(state))
    carry_density.append(norm32(carry_trace))
    clock.append(0.65 * norm32(carry_trace) + 0.35 * norm32(state))
    states.append(state)

return {
    "states": np.array(states, dtype=np.uint32),
    "visible_density": np.array(visible_density),
    "carry_density": np.array(carry_density),
    "clock": np.array(clock),
}

baseline = simulate_carry_history()
soft_reset = simulate_carry_history(reset_step=120, reset_gain=H,
    ↪anti_mode="carry_only")
hard_reset = simulate_carry_history(reset_step=120, reset_gain=1.0,
    ↪anti_mode="hard_flip")

fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(baseline["clock"], label="baseline history")
ax.plot(soft_reset["clock"], label="partial reset at H= /9")
ax.plot(hard_reset["clock"], label="hard reset at 1.0")
ax.axvline(120, linestyle="--")
ax.set_title("Carry-weighted surrogate clock")
ax.set_xlabel("step")
ax.set_ylabel("clock value")
ax.legend()
plt.show()

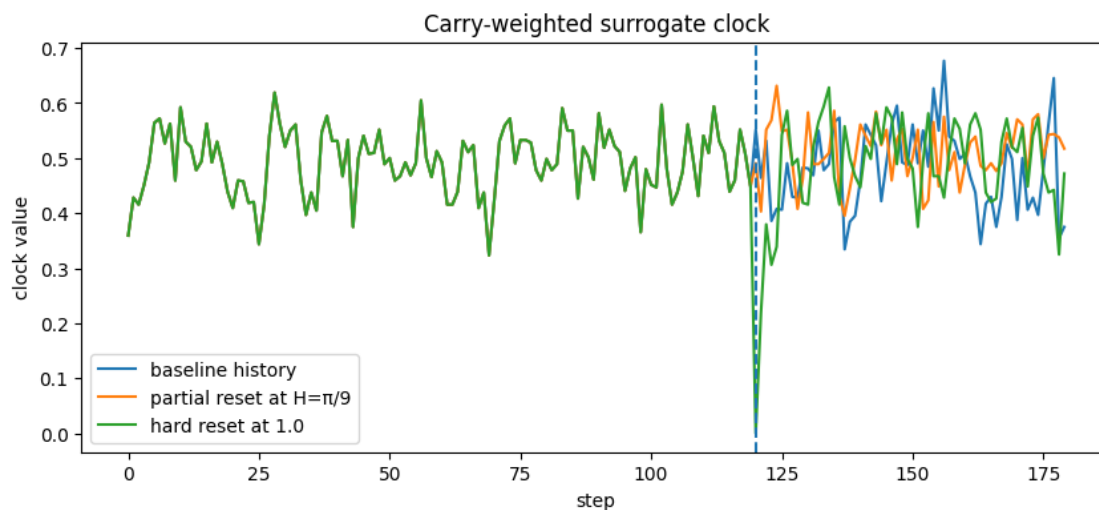
pd.DataFrame({
    "baseline_final_clock": [baseline["clock"][-1]],
    "soft_reset_final_clock": [soft_reset["clock"][-1]],

```

```

"hard_reset_final_clock": [hard_reset["clock"][-1]],
"baseline_final_carry": [baseline["carry_density"][-1]],
"soft_reset_final_carry": [soft_reset["carry_density"][-1]],
"hard_reset_final_carry": [hard_reset["carry_density"][-1]],
}).round(4)

```



```

[3]:      baseline_final_clock  soft_reset_final_clock  hard_reset_final_clock
baseline_final_carry  soft_reset_final_carry  \
0                    0.375                    0.5172                    0.4719
0.375                    0.5938
      hard_reset_final_carry
0                    0.4062

```

1.4 3. Sequence signal and the Sarrus allocation build

The project's bio notebook contains an offline sequence-to-signal pipeline:

1. amino-acid sequence → burial-scale carrier wave
2. autocorrelation at lags 2, 3, 4
3. shuffle-based null
4. helical vs sheet bias summarized as **ZSarrus**

This section preserves that executable logic and runs it on the local Diamond set embedded in the notebook source.

[4]:

```
MJ_BURIAL = {  
    'A': 0.39, 'R': -1.03, 'N': -0.92, 'D': -1.31, 'C': 0.17,  
    'Q': -0.81, 'E': -1.22, 'G': 0.00, 'H': -0.64, 'I': 0.81,  
    'L': 0.70, 'K': -1.15, 'M': 0.44, 'F': 0.92, 'P': -0.31,  
    'S': -0.53, 'T': -0.32, 'W': 0.49, 'Y': 0.26, 'V': 0.69  
}
```

```
DIAMOND_GROUND_TRUTH = [  
    ("2PDD", "E3/E1-PSBD", 41, 9.8),  
    ("2ABD", "ACBP", 86, 6.6),  
    ("256B", "Cyt-b562", 106, 12.2),  
    ("1IMQ", "Im9", 86, 7.3),  
    ("1FNF", "FN3-9", 90, -0.9),  
    ("1WIT", "Twitchin", 93, 0.4),  
    ("1TEN", "Tenascin", 90, 1.1),  
    ("1SHG", "SH3-spectrin", 62, 1.4),  
    ("1SRL", "SH3-src", 64, 4.0),  
    ("1SHF", "SH3-fyn", 67, 4.5),  
    ("1PSF", "PsaE", 69, 3.2),  
    ("1CSP", "CspB-Bs", 67, 7.0),  
    ("1C90", "CspB-Bc", 66, 7.2),  
    ("1G6P", "CspB-Tm", 66, 6.3),  
    ("1MJC", "CspA-Ec", 69, 5.3),  
    ("1LOP", "CypA", 164, 6.6),  
    ("1C8C", "DNA-bp", 63, 7.0),  
    ("1HZ6", "Protein-L", 62, 4.1),  
    ("1PGB", "Protein-G", 57, 6.0),  
    ("1FKB", "FKBP12", 107, 1.5),  
    ("2CI2", "CI2", 64, 3.9),  
    ("1AYE", "ADA2h", 80, 6.8),  
    ("1URN", "U1A", 102, 5.8),  
    ("1APS", "AcP", 98, -1.5),  
    ("1RIS", "S6", 101, 5.9),  
    ("1POH", "HPr", 85, 2.7),  
    ("1DIV", "NTL9", 56, 6.1),  
]
```

```
DIAMOND_SEQUENCES = {  
    "2PDD": "MPKKKMQAFIRKLNMSFKNLQNAKDIMQGFINEKAK",  
    "2ABD": "  
    ↪ \"SQAEDKKAANPASEEMQSAAMSTELTNAEIWKHIQDKEGNGTVEGTWDDFINNIVSQTESKQNLQNLQAEKGLGTDEDTIEDAVKQ\",  
    "256B": "  
    ↪ \"ADLEDLKKHAKISFKDLKDLKSLEPDGQGRITARNADMTSMKQLREQISRLIDREQKLISEEDLGPKDQASRQELQQKIQELINQLREEL\",  
    "1IMQ": "  
    ↪ \"MDNNSIQPYRGKIIVDADLSATRDHDLFLGSEISAGIIATPKEAQKTLSELKHLNQQKQEDISNLKSTANKVFEQLMNEMDAQNLK\",  
    "1FNF": "DPTSYVMDHRGKSYTCTAAYPESGKEVHRISDPTSYVMDHRGKSYTCTAAYPESGKEVHRISV",  
    "1WIT": "EVDIQSKNISATITGQKITFSIPEGKNVTISIPEGKNVTISIPAESDLVIEVNSDPIVLD",  
}
```

```

    "1TEN": "YKLTFTTEIKGQPLTFTIPEGSTLTISGEKTATLTVPEGSTLTINGEKITLKVPEGSTLTIH",
    "1SHG": "MDETGKELVLALYDYQEKS PREVTMKKGDILTLLNSTNKDWWKVEVNDRQGFVPAAYVKKLD",
    "1SRL": "AETIQKLSQKIIHSQISGKAPITFDVPEGQKLISEEDLGPK",
    "1SHF": ␣
    ↪ "MGSSHHHHHSSGLVPRGSHMAEYVRALFDFNGNDEEDLPFKKGDILKVLNEECDQNWEGQLNFK",
    "1PSF": ␣
    ↪ "MKVSTLTIIPAEQDVTVEYKGEKLLIQVENEGDKVTVSIPPEGKVFLKEGDKVTVSIPPEGKIVFLK",
    "1CSP": "MAKVITLEGGKFEVKEGNREKVKASEDLKAKDFEKIENVEGDLQASKNKLITITESGKVVKFKF",
    "1C90": "MAKVITLEGGKFEVKDGNREKVKASEDLKAKDFAKIENVEGDLQASKQKLITITESGKVVKFKF",
    "1G6P": "MAKVITLDGGKFEVKEGTREKVKASEDLKAKDFAKIENVEGDLKASKDKLTITESGKVVKFKF",
    "1MJC": ␣
    ↪ "MGKVNVIADKSFVTGEEGNFKQLAQDMGFQVSGDELGKKLNFKFKEGKPMFQKIAEELGFTVESEGKHIK",
    "1LOP": ␣
    ↪ "MVNPTVFFDIAVDGEPLGRVSFELFADKVPKTAENFRALSTGEKGFYKGSFCFHRIIPGFMCQGGDFTNHNGTGGKSIYGEKFEDENFIL",
    "1C8C": "MKLVKPKELDKQIRSIKPQKIGKLEQTFHPDLTLTEIKQALKPNSIVQNAISKQKSGK",
    "1HZ6": ␣
    ↪ "MEEVTIKANLIFANGYEKQTGEVKKLKAQGEFSATVSQSEGTVLLGEINKNFQMKVKHLKQKIIQELINELKD",
    "1PGB": "MTYKLILNGKTLKGETTTEAVDAATAEKVFKQYANDNGVDGEWYDDATKTFTVTE",
    "1FKB": ␣
    ↪ "MGVQVETISPGDGRTPKRGQTCVVHYTGMLDGGKFDSSRDRNKPFFMLGKQEVIRGWEEGVAQMSVGQRAKLTISPDYAYGATGHPG",
    "2CI2": "MKPKKKLKPTPVKKKKKAPAKKVKDGKVKEKLPEGQKIVNLKEGDKVTVSIPPEGKIVFLK",
    "1AYE": "MSEQNMASEGGQSTGPKSTRELCNIADQNKFITQYYDSEKGGKKLKYRKLQEQLNQLREELKD",
    "1URN": ␣
    ↪ "MSLLNQKNTALQNAQYSVPQSVFSTGKMKDVFINLNRTPRSELENFKRGQVIDGRAVEKGKVPKIVRVGAMREEFNQGKPIHLSLTERE",
    "1APS": ␣
    ↪ "MASGLTAEKAALIDEAEKRLEEIVDKLKGLNIGEGIDIEVKEGDKVTVSIPPEGKVFLKEGDKVTVS",
    "1RIS": ␣
    ↪ "MKVNPSSDLKINTLKIEEGDKVTVSIPPEGKVFLKEGDKVTVSIPPEGKIVFLKEGDKVTVSLPEGKIVFLK",
    "1POH": ␣
    ↪ "MAHKKALVDDFSTMRRIIASKNLAELLAGKDIVTDSEYLTPEAVNQAMKELEKQLGQPVTMSRQPIVKSGDEAFKLIEQEFEDLK",
    "1DIV": "MKVIFLKDVKGMGKKGEIKNVADGYANNFLFKQGLAIEASKLKKVKELKDLHQTAVNIDKK",
}

def seq_to_signal(seq: str) -> np.ndarray:
    arr = np.array([MJ_BURIAL[aa] for aa in seq.upper() if aa in MJ_BURIAL], ␣
    ↪ dtype=float)
    return arr - arr.mean()

def acf_lag(signal: np.ndarray, lag: int) -> float:
    n = len(signal)
    if n <= lag:
        return 0.0
    s0, s1 = signal[:n-lag], signal[lag:]
    denom = np.sqrt(np.dot(s0, s0) * np.dot(s1, s1))
    return float(np.dot(s0, s1) / denom) if denom > 0 else 0.0

```

```

def compute_z_sarrus(seq: str, n_shuffles: int = 300):
    signal = seq_to_signal(seq)
    obs = {lag: acf_lag(signal, lag) for lag in [2, 3, 4]}
    seed = int(hashlib.md5(seq.upper().encode()).hexdigest(), 16) % (2**32)
    rng = np.random.default_rng(seed)
    arr = np.array([MJ_BURIAL[aa] for aa in seq.upper() if aa in MJ_BURIAL])

    null = {lag: [] for lag in [2, 3, 4]}
    for _ in range(n_shuffles):
        s = arr.copy()
        rng.shuffle(s)
        s = s - s.mean()
        for lag in [2, 3, 4]:
            null[lag].append(acf_lag(s, lag))

    z = {}
    for lag in [2, 3, 4]:
        mu, sd = np.mean(null[lag]), np.std(null[lag])
        z[lag] = (obs[lag] - mu) / sd if sd > 0 else 0.0

    z_helix = (z[3] + z[4]) / 2.0
    z_sheet = z[2]
    return {
        "z_sarrus": z_helix - z_sheet,
        "z_helix": z_helix,
        "z_sheet": z_sheet,
        "z2": z[2], "z3": z[3], "z4": z[4],
    }

def run_diamond_build():
    rows = []
    for pdb, name, exp_len, ln_kf in DIAMOND_GROUND_TRUTH:
        seq = DIAMOND_SEQUENCES[pdb]
        used_len = len([aa for aa in seq.upper() if aa in MJ_BURIAL])
        sd = compute_z_sarrus(seq)
        rows.append({
            "pdb": pdb,
            "name": name,
            "length": used_len,
            "ln_kf": ln_kf,
            "z_sarrus": sd["z_sarrus"],
            "z_helix": sd["z_helix"],
            "z_sheet": sd["z_sheet"],
        })
    df = pd.DataFrame(rows).sort_values("z_sarrus", ascending=False).
    ↪reset_index(drop=True)
    r, p = pearsonr(df["z_sarrus"], df["ln_kf"])

```



```

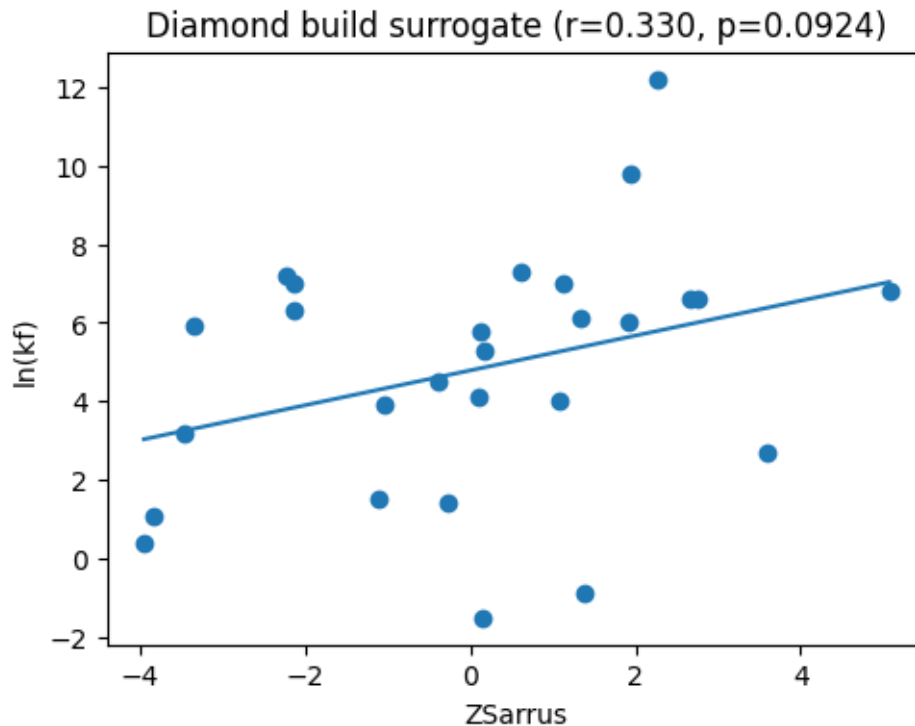
return df, r, p

diamond_df, diamond_r, diamond_p = run_diamond_build()

fig, ax = plt.subplots(figsize=(5.5, 4))
ax.scatter(diamond_df["z_sarrus"], diamond_df["ln_kf"])
b, a, *_ = stats.linregress(diamond_df["z_sarrus"], diamond_df["ln_kf"])
xx = np.linspace(diamond_df["z_sarrus"].min(), diamond_df["z_sarrus"].max(), 100)
ax.plot(xx, b * xx + a)
ax.set_title(f"Diamond build surrogate (r={diamond_r:.3f}, p={diamond_p:.3g})")
ax.set_xlabel("ZSarrus")
ax.set_ylabel("ln(kf)")
plt.show()

diamond_df.head(10).round(4)

```



```

[4]:      pdb      name  length  ln_kf  z_sarrus  z_helix  z_sheet
0  1AYE      ADA2h     64     6.8    5.0741    1.5916   -3.4825
1  1POH       HPr     88     2.7    3.5902   -0.1449   -3.7350
2  1LOP      CypA    152     6.6    2.7525    0.3548   -2.3977
3  2ABD      ACBP     87     6.6    2.6688    0.7805   -1.8883
4  256B  Cyt-b562     92    12.2    2.2598    0.4021   -1.8576

```

5	2PDD	E3/E1-PSBD	41	9.8	1.9503	0.0636	-1.8866
6	1PGB	Protein-G	56	6.0	1.9236	0.2886	-1.6350
7	1FNF	FN3-9	63	-0.9	1.3838	0.3987	-0.9851
8	1DIV	NTL9	61	6.1	1.3234	0.0558	-1.2676
9	1C8C	DNA-bp	60	7.0	1.1293	0.0802	-1.0491

1.5 4. Protein verbs and local perturbation

The protein-folding notebooks repeatedly return to one executable point: the sequence is walked as a **verb**, while the visible backbone is the projection of that verb into 3D.

This section keeps that idea offline and concrete:

- a sequence-specific harmonic signal
- a projected 3D backbone
- compactness / helical periodicity proxies
- a local mutation scan

```
[5]: AA_PHASE = {
    "A": 0.10, "C": 0.20, "D": -0.35, "E": -0.30, "F": 0.40,
    "G": 0.05, "H": -0.10, "I": 0.35, "K": -0.28, "L": 0.30,
    "M": 0.22, "N": -0.18, "P": -0.42, "Q": -0.12, "R": -0.25,
    "S": -0.10, "T": -0.05, "V": 0.28, "W": 0.45, "Y": 0.18,
}

def sequence_backbone(seq: str, base_theta_deg=100.0, base_rise=1.50, bond=3.
↪80):
    theta0 = math.radians(base_theta_deg)
    xyz = np.zeros((len(seq), 3))
    z = 0.0
    ang = 0.0
    radius = math.sqrt((bond**2 - base_rise**2) / (4.0 * (math.sin(theta0 / 2.
↪0)**2)))

    for i, aa in enumerate(seq):
        phase = AA_PHASE.get(aa, 0.0)
        theta = theta0 * (1.0 + 0.15 * phase)
        rise = base_rise * (1.0 - 0.10 * phase)
        ang += theta
        z += rise
        r = radius * (1.0 + 0.08 * phase)
        xyz[i] = [r * math.cos(ang), r * math.sin(ang), z]
    xyz -= xyz.mean(axis=0)
    return xyz

def radius_of_gyration(coords):
    c = coords - coords.mean(axis=0)
    return float(np.sqrt(np.mean(np.sum(c*c, axis=1))))
```

```

def local_step_lengths(coords):
    return np.linalg.norm(np.diff(coords, axis=0), axis=1)

def helical_periodicity(seq: str):
    sig = seq_to_signal(seq)
    return (acf_lag(sig, 3) + acf_lag(sig, 4)) / 2.0

def fold_proxy(seq: str):
    coords = sequence_backbone(seq)
    rg = radius_of_gyration(coords)
    bond_err = float(np.mean(np.abs(local_step_lengths(coords) - 3.8)))
    helix = helical_periodicity(seq)
    compactness = len(seq) / (rg + 1e-9)
    return {
        "Rg": rg,
        "bond_error": bond_err,
        "helical_periodicity": helix,
        "compactness": compactness,
        "coords": coords,
    }

melittin = "GIGAVLKVLTTGLPALISWIKRKRQQ"
collagen_like = "GPPGPPGPPGPPGPPGPPGPPGPP"
scrambled = "QKGLRTLIPAVVLGKIGTSWLKRPQ"

fold_rows = []
for name, seq in [("melittin", melittin), ("collagen_like", collagen_like),
    ↪("scrambled", scrambled)]:
    fp = fold_proxy(seq)
    fold_rows.append({
        "sequence": name,
        "length": len(seq),
        "Rg": fp["Rg"],
        "bond_error": fp["bond_error"],
        "helical_periodicity": fp["helical_periodicity"],
        "compactness": fp["compactness"],
    })

fold_df = pd.DataFrame(fold_rows)

fig = plt.figure(figsize=(10, 4))
for idx, (name, seq) in enumerate([("melittin", melittin), ("collagen_like",
    ↪collagen_like)], start=1):
    ax = fig.add_subplot(1, 2, idx, projection="3d")
    coords = fold_proxy(seq)["coords"]
    ax.plot(coords[:,0], coords[:,1], coords[:,2])

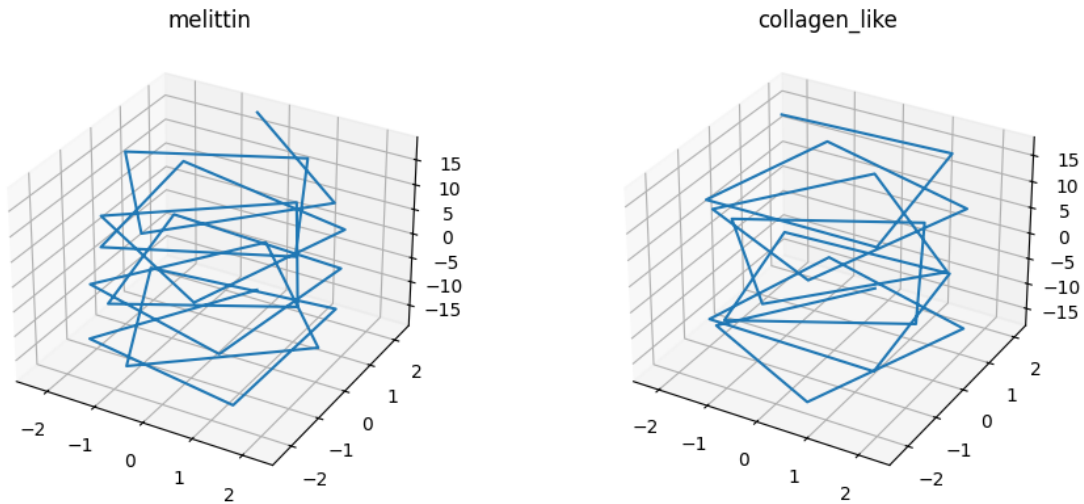
```

```

    ax.set_title(name)
plt.tight_layout()
plt.show()

fold_df.round(4)

```



```

[5]:

```

	sequence	length	Rg	bond_error	helical_periodicity	compactness
0	melittin	26	11.4048	0.1040	0.3349	2.2797
1	collagen_like	24	10.8892	0.1479	0.2593	2.2040
2	scrambled	25	10.9815	0.1087	0.2736	2.2766

```

[6]: def mutation_scan(seq: str, replacement="G"):
    base = fold_proxy(seq)
    rows = []
    for i, aa in enumerate(seq):
        mut = seq[:i] + replacement + seq[i+1:]
        fp = fold_proxy(mut)
        rows.append({
            "position": i + 1,
            "from": aa,
            "to": replacement,
            "d_Rg": fp["Rg"] - base["Rg"],
            "d_periodicity": fp["helical_periodicity"] -
base["helical_periodicity"],
            "d_compactness": fp["compactness"] - base["compactness"],
        })
    return pd.DataFrame(rows)

scan_df = mutation_scan(melittin, replacement="G")

```

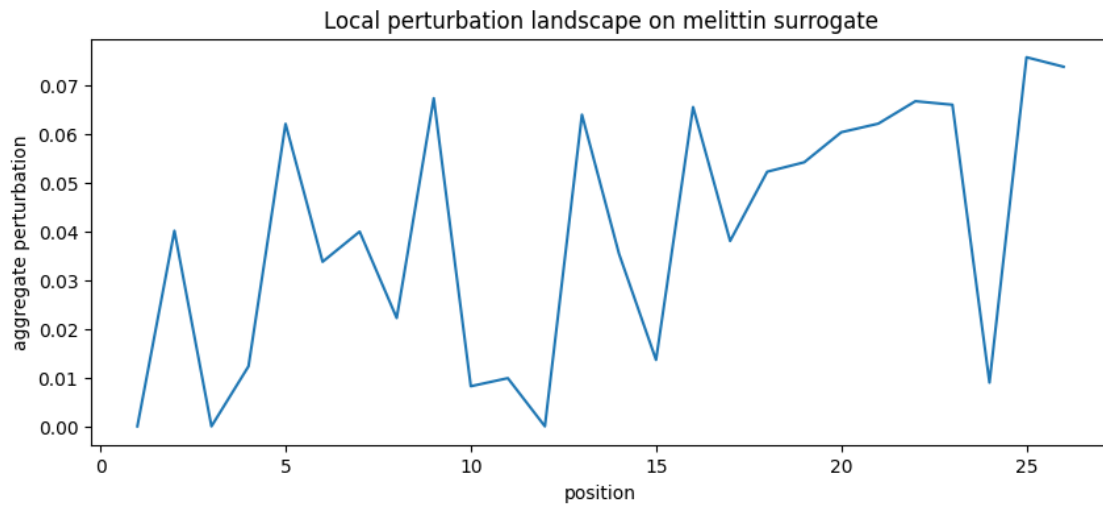
```

scan_df["impact"] = (
    np.abs(scan_df["d_Rg"]) +
    np.abs(scan_df["d_periodicity"]) +
    0.5 * np.abs(scan_df["d_compactness"])
)

fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(scan_df["position"], scan_df["impact"])
ax.set_title("Local perturbation landscape on melittin surrogate")
ax.set_xlabel("position")
ax.set_ylabel("aggregate perturbation")
plt.show()

scan_df.sort_values("impact", ascending=False).head(10).round(4)

```



```

[6]:
  position from to  d_Rg  d_periodicity  d_compactness  impact
24      25    Q  G -0.0029      -0.0726      0.0006  0.0758
25      26    Q  G -0.0014      -0.0723      0.0003  0.0739
8        9    L  G  0.0132      -0.0529     -0.0026  0.0674
21      22    R  G -0.0115      -0.0541      0.0023  0.0668
22      23    K  G -0.0105       0.0545      0.0021  0.0661
15      16    L  G  0.0152      -0.0489     -0.0030  0.0656
12      13    L  G  0.0155      -0.0470     -0.0031  0.0640
20      21    K  G -0.0145      -0.0462      0.0029  0.0622
4        5    V  G  0.0073      -0.0541     -0.0015  0.0621
19      20    I  G  0.0146       0.0443     -0.0029  0.0604

```

1.6 5. Localized reset as a true-mirror problem

The aging paper's central move is not "linear rewind."
It is **localized anti-state injection** at a boundary.

The model below tracks:

- local aged load
- reset field
- protocol mismatch to neighboring tissue
- externalized stress / debt

```
[7]: def lesion_field(n=64, center=(32, 32), sigma=9.0, baseline=0.80,
    ↪lesion_boost=0.18):
    yy, xx = np.mgrid[:n, :n]
    rr2 = (xx-center[0])**2 + (yy-center[1])**2
    lesion = np.exp(-rr2 / (2*sigma**2))
    field = baseline + lesion_boost * lesion
    return np.clip(field, 0.0, 1.0)

def circular_mask(n=64, center=(32, 32), radius=10):
    yy, xx = np.mgrid[:n, :n]
    return ((xx-center[0])**2 + (yy-center[1])**2) <= radius**2

def laplacian2d(arr):
    return (
        np.roll(arr, 1, 0) + np.roll(arr, -1, 0) +
        np.roll(arr, 1, 1) + np.roll(arr, -1, 1) - 4 * arr
    )

def run_reset_simulation(n=64, steps=80, duty=H, radius=10, diffusion=0.08,
    ↪mismatch_gain=0.16):
    state = lesion_field(n=n)
    treat = circular_mask(n=n, radius=radius).astype(float)
    history = []
    mismatch_series = []
    debt_series = []
    local_gain_series = []

    for _ in range(steps):
        before = state.copy()
        state = state - duty * treat * state
        state = state + diffusion * laplacian2d(state)
        state = np.clip(state, 0.0, 1.0)

        mismatch = np.mean(np.abs(laplacian2d(state)) * (1.0 - treat))
        debt = np.mean(np.maximum(0.0, before - state) * treat)
        local_gain = np.mean((before - state) * treat)
        spill = mismatch_gain * mismatch
```

```

state = np.clip(state + spill * (1.0 - treat), 0.0, 1.0)

history.append(state.copy())
mismatch_series.append(mismatch)
debt_series.append(debt)
local_gain_series.append(local_gain)

return {
    "history": history,
    "mismatch": np.array(mismatch_series),
    "debt": np.array(debt_series),
    "local_gain": np.array(local_gain_series),
    "final": history[-1],
}

sim_H = run_reset_simulation(duty=H, radius=10)
sim_full = run_reset_simulation(duty=1.0, radius=10)

fig, axes = plt.subplots(2, 3, figsize=(11, 7))
axes[0,0].imshow(lesion_field()); axes[0,0].set_title("initial aged field")
axes[0,1].imshow(sim_H["final"]); axes[0,1].set_title("final field, duty= /9")
axes[0,2].imshow(sim_full["final"]); axes[0,2].set_title("final field, duty=1.
↪0")

axes[1,0].plot(sim_H["local_gain"], label="local gain, /9")
axes[1,0].plot(sim_full["local_gain"], label="local gain, 1.0")
axes[1,0].set_title("local gain")
axes[1,0].legend()

axes[1,1].plot(sim_H["mismatch"], label="mismatch, /9")
axes[1,1].plot(sim_full["mismatch"], label="mismatch, 1.0")
axes[1,1].set_title("protocol mismatch")
axes[1,1].legend()

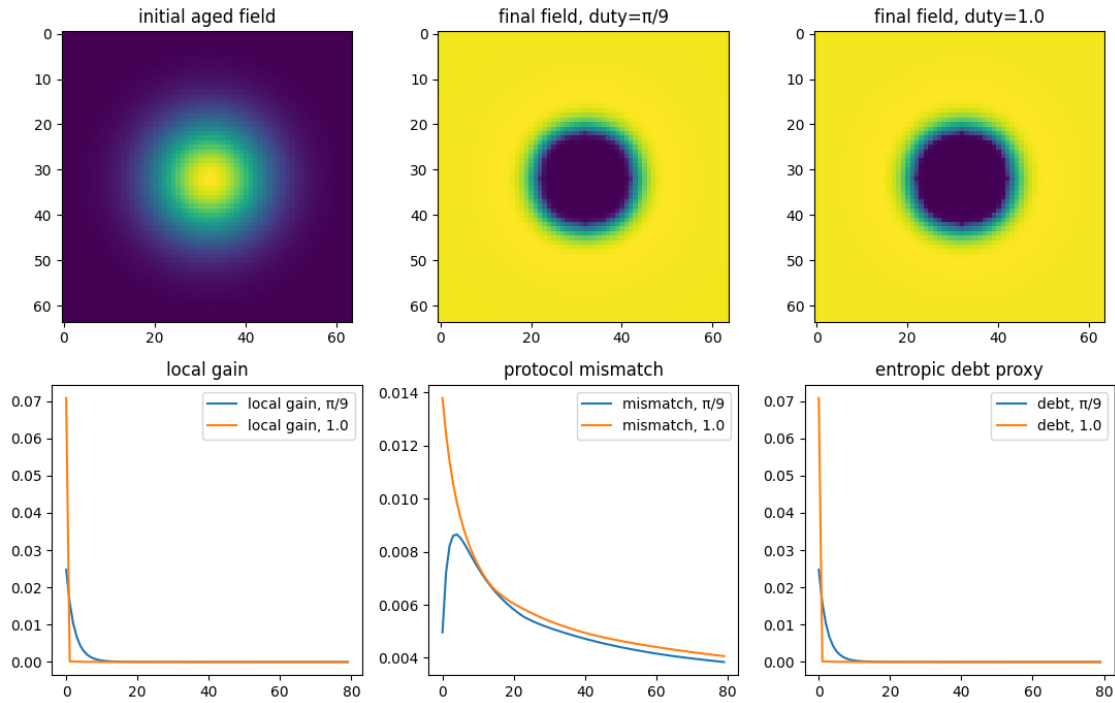
axes[1,2].plot(sim_H["debt"], label="debt, /9")
axes[1,2].plot(sim_full["debt"], label="debt, 1.0")
axes[1,2].set_title("entropic debt proxy")
axes[1,2].legend()

plt.tight_layout()
plt.show()

pd.DataFrame({
    "duty": [H, 1.0],
    "final_local_gain": [sim_H["local_gain"][-1], sim_full["local_gain"][-1]],
    "peak_mismatch": [sim_H["mismatch"].max(), sim_full["mismatch"].max()],
    "mean_debt": [sim_H["debt"].mean(), sim_full["debt"].mean()],

```

```
}).round(4)
```



```
[7]:      duty  final_local_gain  peak_mismatch  mean_debt
0  0.3491          0.0          0.0087    0.0009
1  1.0000          0.0          0.0138    0.0009
```

1.7 6. Duty cycle and coverage sweep

The paper argues that localized success and systemic stability are not the same event. This sweep maps that directly:

- **duty cycle** = intensity of the reset
- **coverage** = fraction of tissue directly hit
- **stability score** = gain minus mismatch minus debt spill

```
[8]: def stability_sweep(duties, coverages, n=64):
    rows = []
    for duty in duties:
        for coverage in coverages:
            radius = int(np.sqrt((coverage * n * n) / np.pi))
            sim = run_reset_simulation(n=n, duty=float(duty),
            ↪radius=max(radius, 1))
            local_gain = sim["local_gain"].sum()
            mismatch = sim["mismatch"].sum()
            debt = sim["debt"].sum()
```



```

        score = local_gain - 0.9 * mismatch - 0.8 * debt
        rows.append({
            "duty": float(duty),
            "coverage": float(coverage),
            "score": float(score),
            "local_gain": float(local_gain),
            "mismatch": float(mismatch),
            "debt": float(debt),
        })
    return pd.DataFrame(rows)

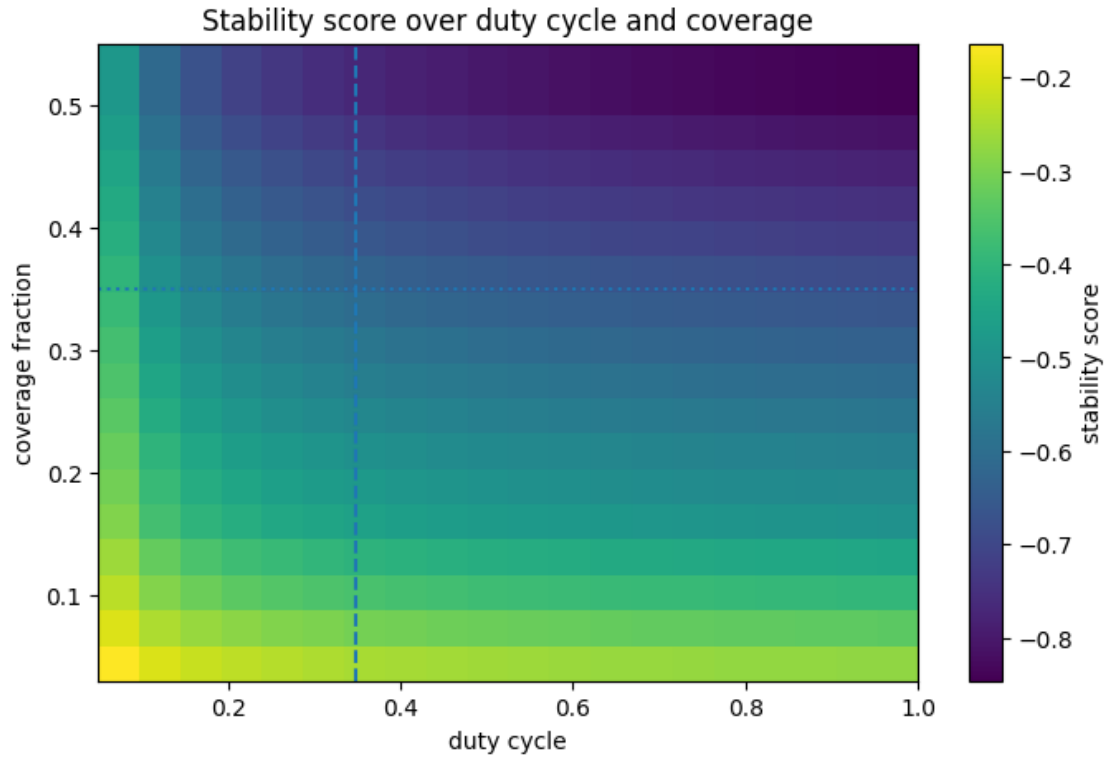
duties = np.linspace(0.05, 1.0, 20)
coverages = np.linspace(0.03, 0.55, 18)
sweep_df = stability_sweep(duties, coverages)

pivot = sweep_df.pivot(index="coverage", columns="duty", values="score")

fig, ax = plt.subplots(figsize=(8, 5))
im = ax.imshow(
    pivot.values,
    aspect="auto",
    origin="lower",
    extent=[pivot.columns.min(), pivot.columns.max(), pivot.index.min(), pivot.
↪index.max()]
)
ax.axvline(H, linestyle="--")
ax.axhline(0.35, linestyle=":")
ax.set_title("Stability score over duty cycle and coverage")
ax.set_xlabel("duty cycle")
ax.set_ylabel("coverage fraction")
plt.colorbar(im, ax=ax, label="stability score")
plt.show()

best_row = sweep_df.sort_values("score", ascending=False).head(1)
best_row.round(4)

```



```
[8]:   duty  coverage   score  local_gain  mismatch   debt
     0  0.05     0.03 -0.1637    0.0226    0.1869  0.0226
```

1.8 7. Subscriber / emitter mismatch

The paper frames treated tissue as a new emitter broadcasting a different temporal protocol than its neighbors.

We can read that mismatch directly as the boundary between a reset region and its untreated surroundings.

```
[9]: def subscriber_emitter_summary(sim, name):
    return {
        "name": name,
        "mean_mismatch": float(sim["mismatch"].mean()),
        "peak_mismatch": float(sim["mismatch"].max()),
        "total_local_gain": float(sim["local_gain"].sum()),
        "total_debt": float(sim["debt"].sum()),
        "gain_to_debt": float(sim["local_gain"].sum() / (sim["debt"].sum() +
↪ 1e-9)),
    }

summary_df = pd.DataFrame([
    subscriber_emitter_summary(sim_H, "duty= /9"),
```

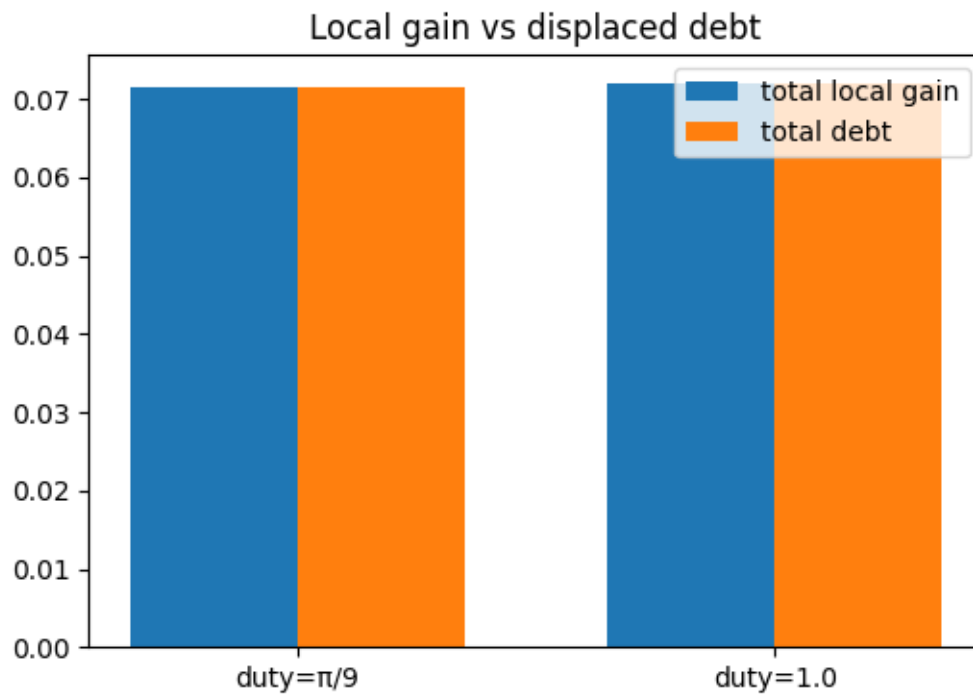
```

    subscriber_emitter_summary(sim_full, "duty=1.0"),
]).round(4)

fig, ax = plt.subplots(figsize=(6, 4))
x = np.arange(len(summary_df))
w = 0.35
ax.bar(x - w/2, summary_df["total_local_gain"], width=w, label="total local_
gain")
ax.bar(x + w/2, summary_df["total_debt"], width=w, label="total debt")
ax.set_xticks(x)
ax.set_xticklabels(summary_df["name"])
ax.set_title("Local gain vs displaced debt")
ax.legend()
plt.show()

summary_df

```



```

[9]:
      name  mean_mismatch  peak_mismatch  total_local_gain  total_debt
gain_to_debt
0  duty= /9      0.0052      0.0087      0.0714      0.0714
1.0
1  duty=1.0      0.0057      0.0138      0.0720      0.0720
1.0

```

1.9 8. A biological hash surrogate

The project documents insist on a stronger claim than analogy: the same operator grammar runs in both domains.

This surrogate stays biological in presentation while keeping the hash view explicit:

- sequence windows are folded into a visible channel
- carry history is preserved separately
- reset targets carry-history load more than identity itself

```
[10]: AA_CODE = {aa: i for i, aa in enumerate("ACDEFGHIKLMNPQRSTVWY", start=1)}

def aa_words(seq: str):
    vals = [AA_CODE.get(aa, 0) for aa in seq]
    words = []
    for i in range(0, len(vals), 4):
        chunk = vals[i:i+4]
        while len(chunk) < 4:
            chunk.append(0)
        word = 0
        for v in chunk:
            word = ((word << 8) | v) & M32
        words.append(word)
    return words

def biological_hash_surrogate(seq: str, reset_at=None, reset_gain=0.0):
    state = 0x6A09E667
    carry = 0
    visible = []
    carry_hist = []

    words = aa_words(seq)
    for idx, w in enumerate(words):
        xor_term, and2_term, summed = kernel_triad_word(state, w)
        carry ^= and2_term
        state = summed

        if reset_at is not None and idx == reset_at:
            carry = int((1.0 - reset_gain) * carry) & M32

        visible.append(norm32(state))
        carry_hist.append(norm32(carry))
    return {
        "state": state,
        "carry": carry,
        "visible_trace": np.array(visible),
        "carry_trace": np.array(carry_hist),
    }
```

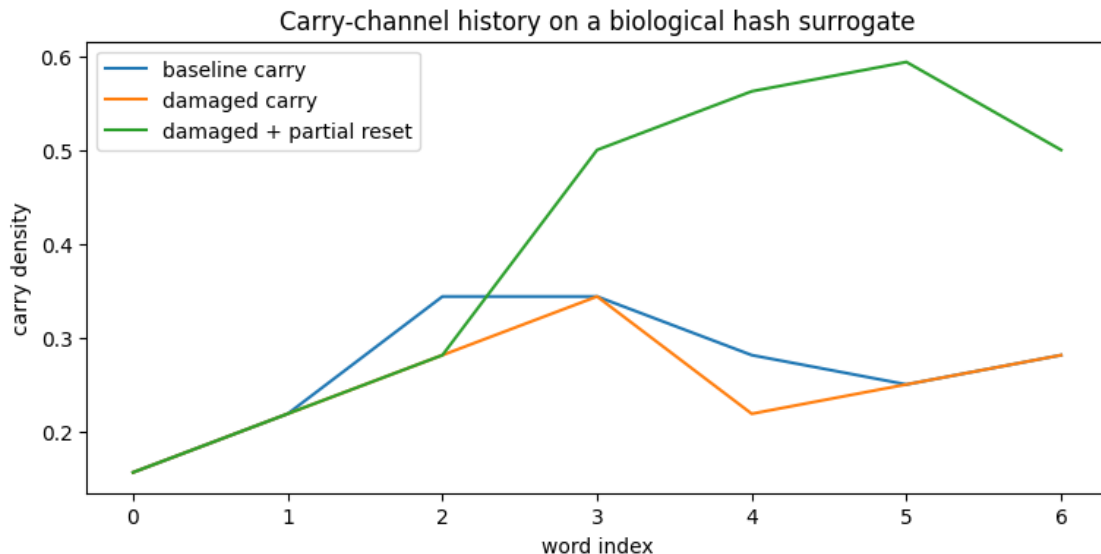
```

seq_base = melittin
seq_damaged = melittin[:10] + "PPPP" + melittin[14:]
base_hash = biological_hash_surrogate(seq_base)
damaged_hash = biological_hash_surrogate(seq_damaged)
reset_hash = biological_hash_surrogate(seq_damaged, reset_at=3, reset_gain=H)

fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(base_hash["carry_trace"], label="baseline carry")
ax.plot(damaged_hash["carry_trace"], label="damaged carry")
ax.plot(reset_hash["carry_trace"], label="damaged + partial reset")
ax.set_title("Carry-channel history on a biological hash surrogate")
ax.set_xlabel("word index")
ax.set_ylabel("carry density")
ax.legend()
plt.show()

pd.DataFrame({
    "case": ["baseline", "damaged", "damaged_reset"],
    "final_visible_density": [
        base_hash["visible_trace"][-1],
        damaged_hash["visible_trace"][-1],
        reset_hash["visible_trace"][-1],
    ],
    "final_carry_density": [
        base_hash["carry_trace"][-1],
        damaged_hash["carry_trace"][-1],
        reset_hash["carry_trace"][-1],
    ],
}).round(4)

```



```
[10]:
```

	case	final_visible_density	final_carry_density
0	baseline	0.5000	0.2812
1	damaged	0.5312	0.2812
2	damaged_reset	0.5312	0.5000

1.10 9. Summary tables

This final section pulls the notebook outputs into a single paper-facing summary.

```
[11]: summary = {
    "H_pi_over_9": H,
    "diamond_build_r": diamond_r,
    "diamond_build_p": diamond_p,
    "best_sweep_duty": float(best_row["duty"].iloc[0]),
    "best_sweep_coverage": float(best_row["coverage"].iloc[0]),
    "best_sweep_score": float(best_row["score"].iloc[0]),
    "soft_reset_peak_mismatch": float(sim_H["mismatch"].max()),
    "hard_reset_peak_mismatch": float(sim_full["mismatch"].max()),
    "soft_reset_mean_debt": float(sim_H["debt"].mean()),
    "hard_reset_mean_debt": float(sim_full["debt"].mean()),
}

summary_df = pd.DataFrame([summary]).round(6)
summary_df
```

```
[11]:
```

	H_pi_over_9	diamond_build_r	diamond_build_p	best_sweep_duty	best_sweep_coverage	best_sweep_score	soft_reset_peak_mismatch	hard_reset_peak_mismatch	soft_reset_mean_debt	hard_reset_mean_debt
0	0.349066	0.330318	0.092418	0.05						
0.03	-0.163715		0.008652							
0		0.013794	0.000893							0.0009

1.10.1 Closing note

The notebook is intentionally paper-facing:

- sequence signal
- fold proxy
- carry-history accumulation
- localized reset
- debt transfer
- coverage / duty-cycle stability
- same operator grammar across the biological and hash readings